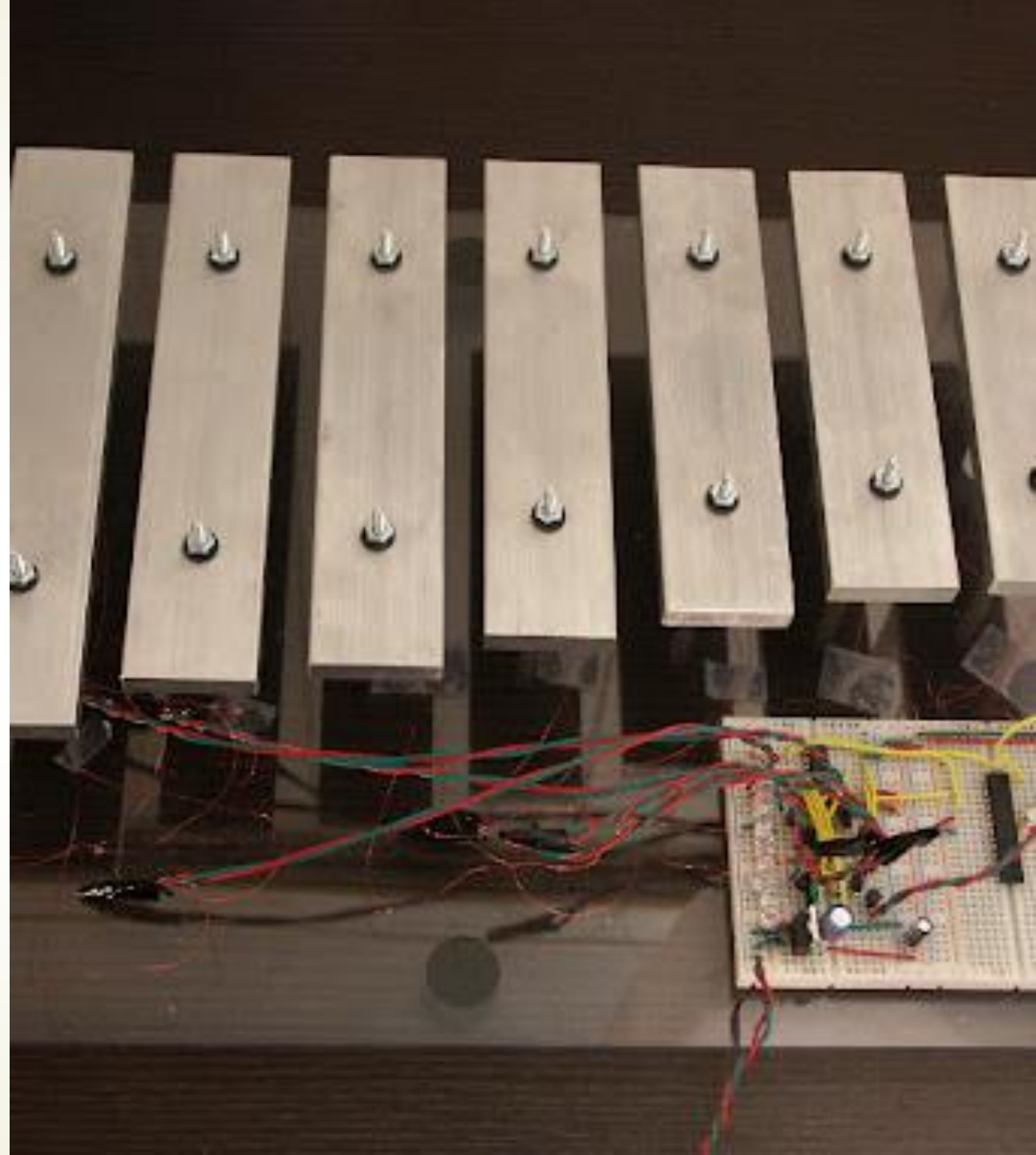


Self Playing Orchestra

CECS 490A

Eric Santana, Dante Jimenez,
Alexander Zepeda, Thaisinge Kour





Problem Description



Problem: Digital music exists but does not interact with real instruments. Music comes from speakers.



Goal: Make a system that allows digital music to control real instruments.



Why?: For people who like the sound of real instruments but don't know how to play.



High Level Solution Overview

Proposed Approach

- ▶ System reads MIDI data.
- ▶ MCU interprets instructions.
- ▶ Electronic drivers activate actuators (Solenoids).
- ▶ Solenoids interact with instrument.

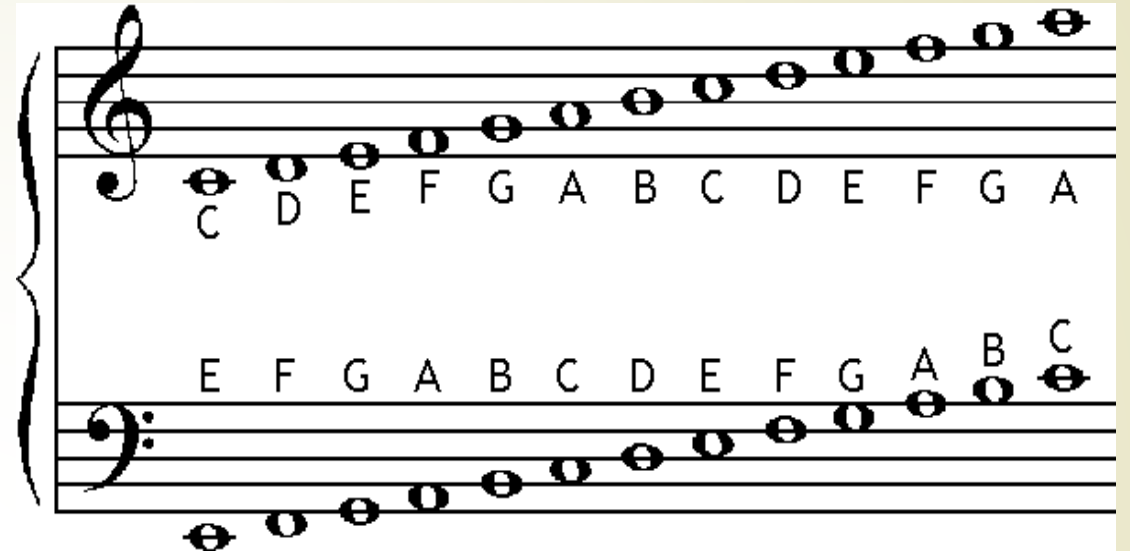
Why is this a reasonable solution?

- MIDI is Standardized
- Solenoids provide reliable motion
- MCU allows for real time control

Instruments

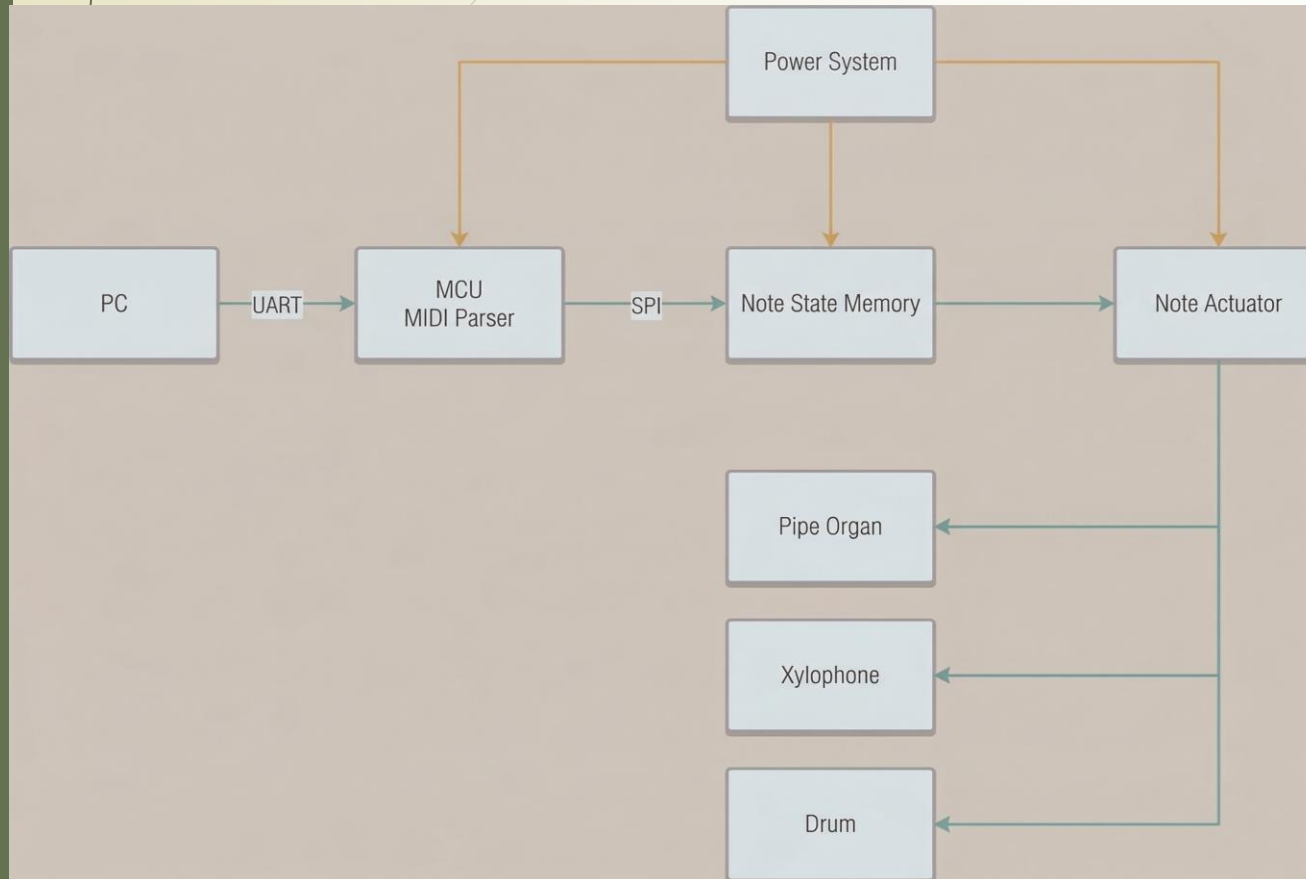
Musical Role	Instrument	Reason
Harmony	3D Printed Pipe Organ	Can play multiple sustained notes (chords)
Melody	Xylophone	Distinct notes, easy to recognize
Rythm	Drum	Provides timing

- This combination allows songs to sound complete while keeping system complexity manageable.
- All instruments will be mounted to a table with wheels for easy transport.



System Architecture

The system converts MIDI data into physical motion



Will be on a custom PCB

- The computer sends MIDI messages to the system using UART.
- The MIDI parser reads the note and instrument channel.
- The system keeps track of which notes should be ON or OFF.
- The actuator turns electrical signals into physical movement using solenoids.
- Each actuator controls one musical note.
- The power system provides voltage for the electronics and solenoids.



PC

Software

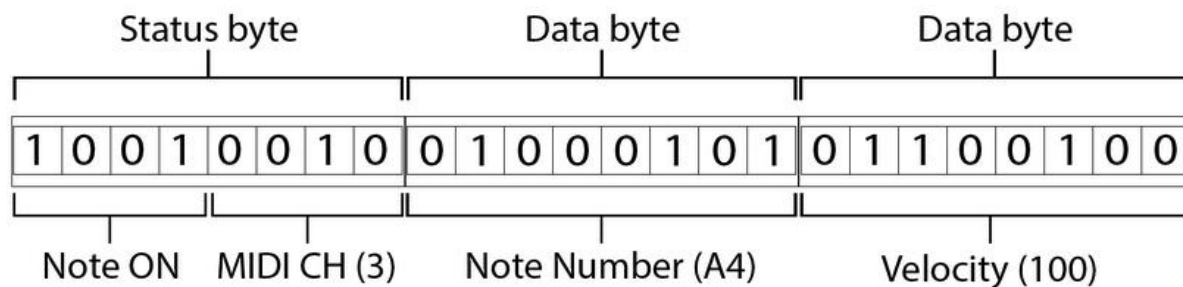
- **MIDI Player:** FL Studio 2025
 - Used to select and play MIDI songs.
- **loopMIDI:**
 - Creates a virtual MIDI port on the PC.
MIDI data is routed from loopMIDI to the COM port.
- **PC Role:**
 - Sends MIDI messages over UART to the microcontroller.

MIDI Parser



- ▶ **Texas Instruments TM4C123 microcontroller**
- ▶ UART receives MIDI data from the computer
- ▶ Interrupt-driven ring buffer stores incoming bytes
- ▶ Parser reads MIDI messages one byte at a time
- ▶ Extracts:
 - Note number
 - Note ON / Note OFF command
 - Instrument channel
- ▶ Decides which actuator should play the note
- ▶ Updates note state memory

Musical Instrument Digital Interface (MIDI)



Music Communication protocol

Octave	C	C#/D	D	D#/E	E	F	F#/G	G	G#/A	A	A#/Bb	B
-1	0	1	2	3	4	5	6	7	8	9	10	11
0	12	13	14	15	16	17	18	19	20	21	22	23
1	24	25	26	27	28	29	30	31	32	33	34	35
2	36	37	38	39	40	41	42	43	44	45	46	47
3	48	49	50	51	52	53	54	55	56	57	58	59
4	60	61	62	63	64	65	66	67	68	69	70	71
5	72	73	74	75	76	77	78	79	80	81	82	83
6	84	85	86	87	88	89	90	91	92	93	94	95
7	96	97	98	99	100	101	102	103	104	105	106	107
8	108	109	110	111	112	113	114	115	116	117	118	119
9	120	121	122	123	124	125	126	127	-	-	-	-

Status Byte

- Upper nibble lets you know if the note can be played or not.

ON

0x09

OFF

0x08

- Lower nibble lets you know the channel number. It decides which instrument plays

Data Byte 1

- Holds the musical note that the system is interacting with.

Data Byte 2

- How loud to play a note. Velocity can be ignored due to solenoid having a fixed force.

Instrument	CH #
Pipe Organ	0
Xylophone	1
Drum	2

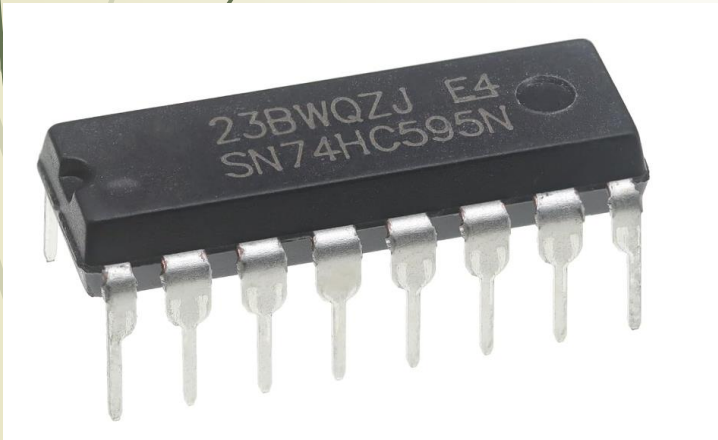
- MIDI supports 128 notes, but our system will have 36 notes to play to reduce complexity and number of actuators.

System Memory

SN74HC595N 8-Bit Shift Registers

Reg 0	b7 G2	b6 F#2/Gb2	b5 F2	b4 E2	b3 D#2/Eb2	b2 D2	b1 C#2/Db2	b0 C2
Reg 1	b7 D#3/Eb3	b6 D3	b5 C#3/Db3	b4 C3	b3 B2	b2 A#2/Bb2	b1 A2	b0 G#2/Ab2
Reg 2	b7 B3	b6 A#3/Bb3	b5 A3	b4 G#3/Ab3	b3 G3	b2 F#3/Gb3	b1 F3	b0 E3
Reg 3	b7 G4	b6 F#4/Gb4	b5 F4	b4 E4	b3 D#4/Eb4	b2 D4	b1 C#4/Db4	b0 C4
Reg 4	b7 x	b6 x	b5 x	b4 Drum	b3 B4	b2 A#4/Bb4	b1 A4	b0 G#4/Ab4

- Uses **SN74HC595N 8-bit shift registers** to control notes
- 36 notes require **5 shift registers (40 total bits)**
- Each bit represents one musical note
- The MCU stores note states in a global variable then sends bytes to shift registers over SPI.
- Bitwise operations turn notes ON or OFF
- Register outputs drive MOSFET gates
- MOSFETs control the solenoids that play each note



Pipe Organ

Xylophone

Drum

Unused

MOSFET & Solenoid



BAOMAIN BM-1564B

Voltage: 24v

Current: 1.4A

Force: 55N



- Shift register output controls the MOSFET gates.
- The MOSFET acts as an electronic switch.
- The MOSFET allows for control of higher current devices.
- When the MOSFET turns ON, current flows through the solenoid.
- The solenoid converts electrical energy into mechanical motion.

Solenoid functions in the system:

- Opens and closes air valves in the pipe organ.
- Strikes xylophone bars to produce sound.
- Hits the drum surface for percussion.

Power System



Main Supply

- Primary supply: **24V DC**.
- Provides power to all 37 solenoids and control electronics.
- If song is complex more solenoids may be on, so Worst-case current $\approx$ **52A**.
- Recommended supply rating: **24V, 60A**.

3.3v Power line

- Buck Converter steps down 24V to 3.3v for control electronics:
 - Powers 8-bit shift registers.
 - Powers TM4C123 microcontroller.

24V Power Line

- Powers Solenoids.



Technical Risks & Challenges

- **Notes outside supported range**

- Notes below 36 or above 71 are shifted into the playable range when possible. Notes that cannot be mapped are ignored.

- **Solenoid voltage spikes (inductive kickback)**

- Solenoids generate high voltage spikes when turned off. Flyback diodes are needed to protect electronics.

- **High current draw**

- If many solenoids activate at once, current demand may be remarkably high.

- **Solution:**

- Use a power supply with sufficient current capacity.
- Limit the number of simultaneous solenoid activations in software.

- **Electrical safety during testing**

- Proper insulation and safe handling procedures are needed when working with exposed wiring.

Thank You

- ▶ If you have a music request (instrumental music preferred), please scan the QR code below:

Enter Song Name

